

The Extended Arsenal

A Companion Volume to Profiling from Zero
The Tools the Core Twenty Chapters Left Out

Prepared for CK | IIT Delhi | Nanopore Project
Prof. Kolin Paul | Summer 2025

Companion Volume · The Extended Arsenal · Chapters 21 to 26

Contents

Chapter 21 What the Core Twenty Left Out: The Gap Analysis

Chapter 22 Modern Dynamic Tracing: eBPF, bpftrace, and Off-CPU

Chapter 23 Profiling the Deep-Learning Stack (Dorado)

Chapter 24 Intel Advisor, PCM, and the HPC Cluster Suite

Chapter 25 Sanitizers, heaptrack, and Static Analysis

Chapter 26 I/O, Python, Continuous, Causal, and the Map

This companion volume assumes the full core book (Chapters 1 to 20): perf, the Valgrind suite, gprof, both Nsight tools, VTune, the Linux system tools, and LIKWID. It covers what those did not.

CHAPTER 21

What the Core Twenty Left Out

This is a companion volume, an extra component to the twenty-chapter book you have already worked through. Its purpose is honest and specific: to tell you what the core book did not cover, to tour the larger universe of profiling tools that exist beyond the ones we studied, and to go deep on the handful of those that genuinely apply to the Nanopore pipeline. The core book gave you a complete, professional toolkit for the common cases. It did not pretend to be exhaustive, and it was not. There is a whole second tier of tools, some niche, some powerful, some that would have been a distraction in the main text but that you should know exist so you can reach for them when the standard toolkit runs out. This volume is that second tier, organized so the brief survey stays brief and the project-relevant tools get the depth they deserve.

Let me start by being precise about what you already have, because the gaps only make sense against the coverage. The core twenty chapters gave you the full perf suite and the roofline model, the entire Valgrind family plus gprof, both Nsight tools for the GPU, Intel VTune with its Top-Down Methodology, the Linux system tools for triage, and LIKWID for measured-peak grounding, all assembled into a decision framework and a report-writing discipline. That is genuinely most of what a working performance engineer uses day to day. What follows is not a correction of that; it is an extension of it.

21.1 The Honest Gap Map

Here, verified against the actual chapter list, are the categories of tooling the core book did not cover. I am laying them out as a map so you can see the shape of what is missing before we explore it, and so you understand that these are deliberate omissions of a second tier, not accidental holes in the first.

- **Modern dynamic tracing.** The core book used perf's counting and sampling subcommands but never touched eBPF, bcc, bpftrace, ftrace, or the perf tracing subcommands (probe, trace, sched, lock). This is the single largest gap, because eBPF is arguably the most important development in Linux observability of the last decade. Chapter 22.
- **Deep-learning framework profiling.** We profiled GPU kernels with Nsight but never connected those kernels back to the model layers that launched them. The PyTorch profiler, Kineto, Holistic Trace Analysis, and the GPU correctness tool compute-sanitizer were all absent, along with practical GPU monitoring like nvidia-smi and DCGM. This matters acutely because Dorado is a LibTorch model. Chapter 23.
- **Automated and commercial CPU tools beyond VTune.** Intel Advisor (which automates the roofline and advises on vectorization), Intel PCM (which reads the uncore counters perf cannot easily reach), AMD uProf, and Oracle's modern gprofng were not covered. Chapter 24.
- **HPC cluster profilers.** The core book was single-machine. The world of multi-node MPI profiling, HPCToolkit, TAU, Score-P, Scalasca, Vampir, Caliper, and Arm MAP, was left out

entirely. This becomes relevant the moment you scale onto the LUNA cluster. Chapter 24.

- **Faster correctness and static analysis.** The compiler sanitizers (AddressSanitizer, ThreadSanitizer, and friends) that catch the same bugs as Memcheck at a fraction of the overhead were absent, as were static instruction-throughput analyzers like llvm-mca and the fast heap profiler heaptrack. Chapter 25.
- **I/O, Python, continuous, and causal profiling.** I/O characterization (Darshan, fio), Python profilers for the pipeline glue (Scalene, py-spy, memray), always-on production profilers (Pyroscope, Parca), and the genuinely novel causal profiler Coz were all out of scope. Chapter 26.

Why These Were Omitted

None of these were omitted because they are bad. They were omitted because the core book optimized for a complete, learnable first toolkit, and adding twelve more tool families would have buried the essential ones under the merely useful. The rule for what made the core book: would a working engineer reach for this weekly? The rule for this companion: is this worth knowing exists, and occasionally worth reaching for? A few of the tools here are weekly-grade for specific situations, especially eBPF and the deep-learning profilers, and those get full depth. The rest get an honest tour with their unique selling point named, so you know what each is for and when it would be the right call.

21.2 How to Read This Volume

The structure follows the gap map. Each chapter takes one category, surveys the tools in it with their unique selling points stated plainly, and then goes deep on whichever tools genuinely apply to the Dorado-and-Kraken-2 pipeline. The depth is not uniform by design. A tool you will actually run on this project gets the same treatment the core book gave perf or Nsight: how it works, how to invoke it, what its output means, and how it connects to the report. A tool that is excellent but irrelevant to genomics on your hardware gets a paragraph naming what it is best at, so you would recognize the situation that calls for it.

I want to set expectations about precision, in the same spirit of honesty the core book kept. For the tools I have run extensively, the detail is concrete. For the tools that are real and well-documented but that I have not personally driven against this exact pipeline, I will describe their documented capabilities and flag where the specifics of integrating them with Dorado or Kraken-2 would need verification on your actual setup. The line between "this is how the tool works" and "this is how I expect it to behave on your pipeline" will be explicit. Profiling is an empirical discipline; a companion volume that overstated its certainty about tools it had not measured on your workload would betray the whole point of the core book, which was to trust measurement over assertion.

21.3 A Quick Map of the Project Relevance

Before the tour, let me tell you which of the coming tools actually matter for the Nanopore work, so you can read with your priorities clear. The genuinely project-relevant additions, the ones worth your time to set up, are these.

- **For Dorado (GPU):** the PyTorch profiler ecosystem and Holistic Trace Analysis, which map GPU kernels back to model operators in a way Nsight alone does not, and compute-sanitizer for GPU correctness. Plus practical GPU monitoring (nvidia-smi, nvidia-top, DCGM) on the LUNA cluster. Chapter 23.
- **For Kraken-2 (CPU):** eBPF and bpftrace for tracing the mmap and page-fault behavior of the 180 GB database and for off-CPU analysis, Intel Advisor for the automated roofline on the Xeon, Intel PCM for per-channel memory bandwidth, and the compiler sanitizers as a faster correctness check than Valgrind. Chapters 22, 24, 25.
- **For the pipeline as a whole:** Darshan or simpler I/O tracing for the disk behavior of reading pod5 signal and the Kraken database, Scalene or py-spy if any orchestration glue is in Python, and the causal profiler Coz for deciding which pipeline stage to optimize for end-to-end speedup. Chapter 26.
- **For scaling on LUNA:** the HPC suite (HPCToolkit and the Score-P family) if you ever run the pipeline across multiple nodes and need MPI-aware, low-overhead, at-scale profiling. Chapter 24.

Everything else in this volume is worth knowing exists but is not on the critical path for this project. The deep dives are reserved for the list above. The rest is the tour. With that map in hand, the companion volume should read as a focused extension rather than an undifferentiated catalog: you will know, as each tool appears, whether to study it or simply file it away.

How This Connects to Your Project

The single most valuable additions this volume makes to the project are two. First, the deep-learning profilers in Chapter 23, which close a real gap in the core book's Dorado analysis: Nsight told you which kernel is hot, but it could not tell you that the kernel belongs to the third Transformer block's attention, and operator-level attribution is exactly what makes a deep-learning performance recommendation actionable. Second, eBPF in Chapter 22, which gives you a low-overhead, production-safe way to trace Kraken-2's page-fault and mmap behavior on the real 180 GB database, something Valgrind is too slow to do on the full input and perf only partially captures. Those two close the most consequential gaps. The rest sharpen the edges.

What We Just Learned

The core twenty chapters cover perf, the Valgrind suite, gprof, both Nsight tools, VTune with TMA, the Linux system tools, LIKWID, and the report and decision frameworks. That is most of what a working engineer uses daily.

The verified gaps fall into six categories: modern dynamic tracing (eBPF and friends), deep-learning framework profiling, automated and commercial CPU tools, HPC cluster profilers, faster correctness and static analysis, and I/O, Python, continuous, and causal profiling.

These were omitted to keep the core toolkit learnable, not because they lack value. This volume tours them with their unique selling points and goes deep only on the ones that apply to the Nanopore pipeline.

The project-relevant additions are the deep-learning profilers and compute-sanitizer for Dorado, eBPF and Intel Advisor and PCM and the sanitizers for Kraken-2, I/O and Python and causal profilers for the pipeline, and the HPC suite if scaling onto LUNA.

The depth is deliberately uneven: project-relevant tools get full treatment, the rest get an honest tour. Where integration specifics with your exact pipeline are unverified, that uncertainty will be stated explicitly, keeping faith with the core book's measurement-over-assertion discipline.

Before You Move On

You now have the map of what is missing and which missing pieces matter for the project. Read the deep-dive sections as carefully as you read the core book; read the tour sections to build recognition, not mastery.

The next chapter takes on the largest gap: modern dynamic tracing with eBPF, bpftrace, and the perf tracing subcommands. This is the most important single addition to your toolkit, a way to ask almost any question about a running system with minimal overhead and no recompilation, and it is directly applicable to understanding how Kraken-2's enormous memory-mapped database actually behaves under load.

CHAPTER 22

Modern Dynamic Tracing: eBPF

This is the most important tool the core book left out, so it gets the most depth. Everything you learned about perf was about counting and sampling: how many cache misses, where was the program when sampled. Dynamic tracing answers a different and often more powerful kind of question: what exactly happened, when, and why, traced from inside the running kernel, with almost no overhead and no recompilation. The technology is eBPF, the extended Berkeley Packet Filter, which lets you load small safe programs into the running Linux kernel that fire on events you choose, function calls, system calls, page faults, network packets, and collect exactly the data you want. It is the foundation of modern Linux observability, and it answers questions that perf and Valgrind structurally cannot.

Let me motivate it with a question the core toolkit cannot answer well. When Kraken-2 stalls, is it stalled on the CPU computing, or is it stalled off the CPU, blocked waiting for a page fault to be serviced from the 180 GB database, or waiting on disk I/O? perf sampling sees the program when it is on the CPU; it is largely blind to time spent blocked and off the CPU, because a blocked thread is not running to be sampled. This off-CPU time can dominate a memory-mapped workload like Kraken-2, where threads constantly block waiting for database pages to fault in. eBPF can trace exactly this: it can record every time a thread goes off the CPU, why, and for how long, building a complete picture of blocked time that sampling misses entirely. That single capability, off-CPU analysis, would have been worth a chapter on its own.

22.1 What eBPF Is

eBPF is a virtual machine inside the Linux kernel that runs small, verified programs in response to events. You write a tiny program, the kernel checks it for safety (it cannot crash the kernel, cannot loop forever, cannot access arbitrary memory), and then attaches it to a hook: a kernel function entry, a system call, a tracepoint, a user-space function, a hardware event. When the hook fires, your program runs, reads the relevant data, and stores it in a map that user space can read. The overhead is tiny because the program runs in the kernel at native speed and only does the small amount of work you specified.

The genius of eBPF for profiling is that it is programmable and dynamic. perf gives you fixed counters and fixed sampling. eBPF lets you define a custom measurement: trace every mmap call and record its size and the stack that called it, or measure the latency distribution of every block-device I/O, or count how long each thread spends blocked and on what. You are not limited to the events the tool's authors anticipated; you write the probe for the question you have. And it is dynamic: you attach and detach probes on a running production process with no restart and no recompilation, which is why it is the basis of modern always-on production observability.

The Core Distinction

perf and Valgrind ask predefined questions: how many cache misses, where is the time, what does the cache do. eBPF lets you ask arbitrary custom questions of a running system: trace this specific event, record exactly this data, aggregate it this way, all with kernel-native overhead and no recompilation. It is the difference between a set of fixed instruments and a programmable instrument you configure for the question at hand. This is why it filled the biggest gap: it is not one more fixed tool, it is a way to build the tool you need.

22.2 The eBPF Toolset: bcc and bpftrace

You rarely write raw eBPF. Two front ends make it practical, and they sit at different points on the convenience-versus-power spectrum.

bcc, the BPF Compiler Collection, is a toolkit of ready-made tracing tools plus a Python framework for writing your own. The ready-made tools are the treasure: dozens of small programs, each answering one question, written by the observability community. *offcputime* traces off-CPU blocked time. *biolatency* shows block-device I/O latency as a histogram. *funclatency* times any kernel or user function. *cachestat* shows page-cache hit rates. You run them like any command and they just work. This is the layer you reach for first, because someone has usually already written the tool for your question.

bpftrace is a high-level tracing language, deliberately reminiscent of awk, for writing one-line or short custom tracers when no ready-made bcc tool fits. You express the probe, the data to collect, and the aggregation in a compact syntax, and bpftrace compiles it to eBPF and runs it. It is the layer for ad-hoc custom questions, the programmable instrument in its most accessible form.

```
# install on Ubuntu / WSL2 (needs a recent kernel with BTF)
sudo apt install bpfcc-tools bpftrace linux-headers-$(uname -r)

# bcc ready-made tools live with a -bpfcc suffix on Ubuntu:
sudo offcputime-bpfcc -p $(pgrep -n kraken2) 10 # off-CPU stacks, 10s
sudo biolatency-bpfcc 10 1 # block I/O latency histogram
sudo cachestat-bpfcc 1 # page cache hit rate per sec
```

bash

A caution for the lab WSL2 machine: eBPF needs a reasonably modern kernel, ideally with BTF (BPF Type Format) for the most convenient tools. Recent WSL2 kernels support eBPF, but older ones may not, and some probe types are restricted under virtualization. On the LUNA cluster and the sauron machine, native Linux kernels generally give you the full capability. Where eBPF is unavailable, the perf tracing subcommands in Section 22.5 provide a partial fallback. Verify with a simple bpftrace one-liner before relying on it for the project.

22.3 Off-CPU Analysis: The Killer Application for Kraken-2

Here is the eBPF capability that directly addresses a question the core book could not. On-CPU profiling, which is what `perf record` does, shows where the program spends time while running. Off-CPU profiling shows where the program spends time while *blocked*, not running, waiting for something: a lock, a page fault, disk I/O, a system call to return. For a compute-bound program these are different views of nearly the same thing. For a memory-mapped, I/O-touching program like Kraken-2, off-CPU time can be the majority of the wall-clock time, and on-CPU profiling is blind to all of it.

The `bcc` tool `offcputime` traces every context switch away from the CPU, records the stack that was running and how long the thread stayed blocked, and aggregates the result as a set of stacks ranked by total off-CPU time. Run it on Kraken-2 and you see exactly where the threads block and for how long. If the dominant off-CPU stack is in the page-fault handler, that is the 180 GB database faulting in from disk or being reclaimed under memory pressure, the smoking gun for a memory-capacity problem. If it is in a lock, that is contention among the classification threads. If it is in a read system call, the input file is the bottleneck. None of these show up in a `perf record` on-CPU flame graph, because a blocked thread is not on the CPU to be sampled.

```
# capture Kraken-2's off-CPU stacks for 30 seconds
sudo offcputime-bpfcc -p $(pgrep -n kraken2) -f 30 > kraken_offcpu.folded

# render as an off-CPU flame graph (same FlameGraph scripts as Chapter 4)
./FlameGraph/flamegraph.pl --color=io --title="Kraken-2 Off-CPU" \
    kraken_offcpu.folded > kraken_offcpu.svg
```

bash

The off-CPU flame graph reads the same way as the on-CPU one from Chapter 4, width is total blocked time, but now the wide plateaus are where the program waited rather than where it computed. Putting the on-CPU and off-CPU flame graphs side by side gives the complete time accounting: this much time computing the hash lookups, this much time blocked waiting for pages to fault in. For Kraken-2 that side-by-side is genuinely revealing, because it answers whether the bottleneck is the cache-miss latency while running (an on-CPU problem the core book analyzed) or the page-fault and I/O latency while blocked (an off-CPU problem only eBPF surfaces clearly). The honest diagnosis often involves both, and only with off-CPU analysis can you weigh them.

Key Insight

On-CPU profiling shows where a program computes; off-CPU profiling shows where it waits. `perf record` sees only the first because a blocked thread cannot be sampled. For a memory-mapped, I/O-bound workload like Kraken-2 with its 180 GB faulting database, the waiting can dominate the wall-clock time, and missing it means misdiagnosing the program as compute-bound when it is actually blocked half the time. eBPF off-CPU analysis is the only practical way to see this on a real full-scale run, because Valgrind is too slow and `perf` is structurally blind to blocked time.

22.4 Tracing the Database mmap and Page Faults

Beyond off-CPU time, eBPF lets you trace the specific events that govern Kraken-2's memory behavior. A short bpftrace script can count and time page faults, attribute them to stacks, and watch the mmap calls that map the database. This complements the Massif page-level view from the core book and the strace mmap trace, but with the ability to aggregate live over a full run rather than a small input.

```
# count page faults by user stack for a process, live
sudo bpftrace -e '
  software:page-faults:1 /pid == $1/ {
    @faults[ustack] = count();
  }' $(pgrep -n kraken2)

# histogram of how long each block I/O takes (the DB paging in)
sudo bpftrace -e '
  tracepoint:block:block_rq_issue { @start[args->sector] = nsecs; }
  tracepoint:block:block_rq_complete /@start[args->sector]/ {
    @usecs = hist((nsecs - @start[args->sector]) / 1000);
    delete(@start[args->sector]);
  }'
```

The first script attributes page faults to the stacks that trigger them, telling you which part of Kraken-2's code is faulting in database pages. The second builds a latency histogram of the underlying block I/O, telling you whether the pages are coming from a fast SSD (microseconds) or a slow path (milliseconds), which directly affects how much the page faults cost. Together they quantify the database-paging behavior that is the heart of the Kraken-2 memory story, and they do it on the real database at full scale, which is exactly where Valgrind's overhead made it impractical. This is the data that would let the report state, with numbers, how much of Kraken-2's time goes to faulting the database in versus computing on it once it is resident, and therefore how much the Hot-K-mer cache, which keeps hot data resident and local, could recover.

22.5 The perf Tracing Subcommands and ftrace

eBPF is the modern path, but perf itself has tracing subcommands the core book did not cover, and they are a useful fallback where eBPF is restricted, such as some virtualized environments. These extend the perf you already know into the tracing domain.

- **perf top** is a live, continuously updating version of perf report, like the Unix top command but for functions. It shows the current hotspots in real time, which is handy for watching a long-running process like Kraken-2 and seeing the hot function emerge live without waiting for the run to finish.
- **perf trace** is a perf-based strace replacement that traces system calls with lower overhead than strace, useful for the same mmap-tracing questions on a process where strace's overhead would distort timing.
- **perf probe** lets you insert dynamic tracepoints at arbitrary function entries or even source lines, then record on them, the perf analogue of an eBPF kprobe, so you can sample specifically when a chosen function runs.

- **perf sched** analyzes scheduler behavior, including how long threads wait to run, which is a partial on-CPU-scheduling view of the same blocked-time question off-CPU analysis answers more completely.
- **perf lock** analyzes kernel lock contention, relevant if Kraken-2's threads contend on kernel locks during heavy paging.

Beneath perf's tracing and beneath eBPF's tracepoint probes sits **ftrace**, the kernel's built-in function tracer, usually driven through the *trace-cmd* front end and visualized with KernelShark. ftrace can trace every kernel function call, which is more than you usually want, but it is the ground truth for understanding what the kernel is doing on behalf of your process, and it predates and underlies much of the higher-level tooling. For the project you will rarely drop to raw ftrace, but knowing it is the foundation explains where the higher tools get their data. The older dynamic-tracing systems SystemTap and, on non-Linux platforms, DTrace, occupy the same conceptual space as eBPF and are worth recognizing by name, but on modern Linux eBPF has largely superseded them and is the right default.

How This Connects to Your Project

eBPF is the most consequential addition to the Kraken-2 analysis. The core book diagnosed Kraken-2 as memory-bound from its cache-miss rate and IPC, all on-CPU measurements. eBPF off-CPU analysis answers the question those measurements leave open: how much of Kraken-2's wall-clock time is spent blocked waiting for database pages to fault in, versus running and missing cache. On the real 180 GB database at full scale, where Valgrind is far too slow and perf sees only on-CPU time, a thirty-second offcputime capture and a page-fault bpftrace script give you the blocked-time accounting directly.

That accounting sharpens the Hot-K-mer cache argument considerably. If off-CPU analysis shows that a large fraction of time is page-fault wait on the database, the cache's value is not just reducing cache misses (the on-CPU story) but reducing page faults by keeping hot k-mers resident (the off-CPU story). The report becomes stronger: it can show, with eBPF numbers, both the on-CPU cache-miss cost and the off-CPU paging cost, and argue that the cache addresses both. For Dorado, eBPF can trace the host-side I/O of reading pod5 signal files, revealing whether disk reads are blocking the pipeline and starving the GPU, which connects to the timeline gaps the core book's Nsight Systems chapter flagged. The caveat: confirm eBPF works on the specific lab WSL2 kernel before relying on it there; on LUNA's native Linux it should be fully available.

Common Pitfall

Diagnosing a memory-mapped workload as compute-bound from on-CPU profiling alone. `perf record` cannot see blocked time, so a program that spends half its wall-clock blocked on page faults looks compute-bound in an on-CPU flame graph. Always run off-CPU analysis on an I/O- or mmap-heavy workload.

Assuming eBPF works everywhere. It needs a modern kernel and is restricted in some virtualized environments. Verify with a trivial `bpfftrace` one-liner before relying on it, especially on the lab WSL2 machine.

Writing custom `bpfftrace` when a ready-made `bcc` tool already exists. The `bcc` toolkit has dozens of curated tools; check there first before writing your own probe.

Confusing `perf`'s counting subcommands with its tracing subcommands. `perf stat` and `record count` and `sample`; `perf trace`, `probe`, and `sched trace` events. They answer different questions, and the tracing side overlaps with eBPF's territory.

What We Just Learned

eBPF runs small verified programs inside the kernel in response to events, letting you ask arbitrary custom questions of a running system with kernel-native overhead and no recompilation. It is a programmable instrument, not a fixed one, which is why it filled the largest gap.

The practical front ends are `bcc` (a toolkit of ready-made tracers plus a framework) and `bpfftrace` (a high-level language for custom one-liners). Reach for a ready-made `bcc` tool first; write `bpfftrace` when none fits.

Off-CPU analysis is the killer application for Kraken-2. It traces blocked time that on-CPU sampling cannot see, revealing whether the program waits on page faults, locks, or I/O. For a memory-mapped workload this blocked time can dominate.

eBPF can trace the database `mmap` calls and page faults directly, attributing them to stacks and building I/O latency histograms, quantifying the paging behavior on the real database at full scale where Valgrind is too slow.

The `perf` tracing subcommands (`top`, `trace`, `probe`, `sched`, `lock`) extend `perf` into tracing and serve as a fallback where eBPF is restricted. `ftrace` is the underlying kernel function tracer; `SystemTap` and `DTrace` are the older equivalents eBPF has largely superseded on Linux.

For the project, eBPF off-CPU and page-fault analysis strengthens the Kraken-2 diagnosis by adding the blocked-time accounting the core book's on-CPU measurements left open, and it can trace Dorado's host-side I/O. Verify eBPF availability on the lab WSL2 kernel first.

Before You Move On

You now have the most important missing capability: programmable, low-overhead, off-CPU-aware dynamic tracing, directly applicable to understanding how Kraken-2's database actually behaves under load on real input.

The next chapter closes the most consequential Dorado gap: the deep-learning framework profilers that map GPU kernels back to model operators, plus the GPU correctness tool compute-sanitizer and the practical monitoring tools for the LUNA cluster. Nsight told you which kernel is hot; the next chapter tells you which model layer that kernel belongs to, which is what makes a deep-learning optimization recommendation actionable rather than abstract.

CHAPTER 23

Profiling the Deep-Learning Stack

Nsight Compute told us which GPU kernel dominates Dorado's basecalling. It could not tell us that the kernel is the attention projection in the third Transformer block, or that another hot kernel is the convolutional stem, or that a third is the CTC decoder. Nsight sees kernels; it does not see the model. For a hand-written CUDA program like our `matmul_cuda` benchmark, kernel-level is all there is, and Nsight is complete. But Dorado is not hand-written CUDA. It is a deep neural network expressed in a framework, LibTorch, the C++ form of PyTorch, and the framework launches hundreds of kernels on your behalf, each one an implementation detail of some model operation. To optimize Dorado you need to connect the hot kernels back to the model operators that launched them, and that mapping is exactly the gap the deep-learning profilers fill. This chapter is the most consequential addition to the Dorado analysis.

Let me make the gap concrete. Suppose Nsight tells you a particular GEMM kernel consumes thirty percent of GPU time. That is useful, but the actionable question is which model operation issues that GEMM. If it is the attention score computation, the fix might be a fused attention kernel or a smaller key-value cache. If it is a feed-forward layer, the fix is different. If it is the same GEMM launched from many layers, the fix is architectural. Nsight gives you the kernel; the framework profiler gives you the operator, the layer, the call path through the model. Operator-level attribution is what turns "this kernel is hot" into "the attention in the encoder is hot," and only the latter is something you can act on at the model level.

23.1 The PyTorch Profiler and Kineto

PyTorch ships a profiler, accessible in Python as `torch.profiler`, whose backend is a library called Kineto. Kineto does two things at once: it records the framework's own operator events (this tensor operation, that layer's forward pass) on a CPU timeline, and it simultaneously captures the GPU kernel activity via CUPTI, the CUDA Profiling Tools Interface that underlies Nsight as well. Because it captures both at once and correlates them, it can tell you, for each GPU kernel, which framework operator launched it. That correlation is the unique selling point: it stitches the model's logical operations to the GPU's physical kernels, which neither Nsight nor a pure framework trace does alone.

The honest complication for Dorado is that Dorado is C++ LibTorch, not Python PyTorch, so you do not drive it from `torch.profiler`'s Python interface. There are two practical paths, and which applies depends on Dorado's build, so verify against your version. The first path: LibTorch emits NVTX ranges for its operators when profiling is enabled, and those ranges show up in Nsight Systems exactly as the custom NVTX ranges from the core book did, giving you operator names on the GPU timeline without any separate tool. The second path: the Kineto profiler can be driven from C++ as well, and PyTorch-based applications can be built to emit Kineto traces; whether Dorado exposes this directly or would need a small patch to its source is something to check in its codebase. Either

way, the goal is the same, operator-to-kernel attribution, and the most reliable route for an unmodified Dorado binary is the NVTX-in-Nsight-Systems path, which requires no code changes and uses the Nsight skills you already have.

```
# the reliable, no-code-change path: capture LibTorch's operator NVTX
# ranges in Nsight Systems, so the GPU timeline is labeled by operator
nsys profile --trace=cuda,nvtx \
  --output=dorado_ops \
  dorado basecaller hac signal.pod5 --output-dir results/

# in the timeline, the NVTX rows now show operator-level names
# (aten::matmul, attention, conv, etc.) aligned with the GPU kernels
```

bash

For a model you do control in Python, the torch.profiler usage is straightforward and worth knowing because the trace it produces is what the next section's analysis tool consumes. You wrap the inference in a profiler context, request both CPU and CUDA activities, and export a Chrome-format trace.

```
# Python torch.profiler pattern (for a PyTorch model you drive directly)
import torch
from torch.profiler import profile, ProfilerActivity

with profile(activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
             record_shapes=True, with_stack=True) as prof:
    model(input_batch)    # the inference to profile

# operator-level table, sorted by total CUDA time
print(prof.key_averages().table(sort_by="cuda_time_total", row_limit=20))
# export a trace for Holistic Trace Analysis or chrome://tracing
prof.export_chrome_trace("dorado_trace.json")
```

C

The key_averages table is the deep-learning analogue of perf report and nsys stats: a ranked list, but ranked by operator rather than by function or kernel, with both the CPU operator time and the correlated CUDA kernel time. That table directly answers the question Nsight could not: which model operator costs the most GPU time. For a Transformer like Dorado's, you would expect the attention and the linear projections to dominate, and the table would confirm it operator by operator.

23.2 Holistic Trace Analysis

A single profiler trace is rich but hard to read by eye, especially for a model with many layers running many iterations. Holistic Trace Analysis, abbreviated HTA, is an open-source library that consumes PyTorch profiler traces and computes higher-level analyses: how much time is spent in computation versus communication versus idle, which kernels dominate, how well the GPU is utilized over time, and where the gaps are. Its unique selling point is turning the raw trace into the diagnostic summaries you actually want, the temporal breakdown and the kernel and operator rankings, without manual timeline reading.

For Dorado, the most valuable HTA analyses are the temporal breakdown, which tells you what fraction of wall-clock time the GPU was computing versus idle versus waiting, and the kernel breakdown, which ranks kernels and groups them by the operator type that launched them. The temporal breakdown connects directly to the Nsight Systems timeline gaps from the core book: if HTA reports the GPU was idle a significant fraction of the time, that quantifies the starvation the timeline showed qualitatively, and points at the host-side data loading as the fix. The kernel-by-operator breakdown is the operator-level attribution that makes the optimization actionable. HTA is a Python library you run on an exported trace, so it applies cleanly when you have a trace from a Python-driven model and applies to Dorado to the extent you can produce a compatible trace, again something to verify against Dorado's profiling support.

Key Insight

Nsight gives you kernel-level truth; the framework profilers give you operator-level attribution. The two are complementary, not competing. The ideal Dorado workflow uses both: Nsight Compute to dissect the microarchitecture of the hot kernel (memory-bound or compute-bound, occupancy, stalls) and the PyTorch profiler or NVTX-labeled Nsight Systems trace to know which model operator that kernel belongs to. The kernel analysis tells you how to fix the kernel; the operator attribution tells you which part of the model to fix. A deep-learning performance report needs both layers, and the core book gave you only the first.

23.3 compute-sanitizer: GPU Correctness

The core book gave you Memcheck and Helgrind for CPU correctness, the principle that a fast wrong answer is worse than a slow right one. The GPU has the same hazard and the same need, and the tool is compute-sanitizer, NVIDIA's GPU correctness checker that replaced the older cuda-memcheck. It catches the GPU analogues of the bugs Valgrind catches on the CPU, and it is essential before trusting any GPU result, including Dorado's basecalls if you ever modify its kernels.

compute-sanitizer has four sub-tools, each targeting a class of GPU bug, and together they cover the correctness questions that no performance profiler addresses.

- **memcheck:** detects invalid global-memory accesses, out-of-bounds reads and writes, and misaligned accesses, the GPU analogue of Memcheck's addressability checking. An out-of-bounds access in a CUDA kernel often does not crash; it silently corrupts, which is exactly the dangerous case.
- **racecheck:** detects shared-memory data races, where threads in a block access the same shared-memory location without proper synchronization, the GPU analogue of Helgrind. The tiled matmul's `__syncthreads` calls exist precisely to prevent such races; racecheck verifies they are correct.
- **initcheck:** detects reads of uninitialized global memory, the GPU analogue of Memcheck's definedness checking. Reading uninitialized device memory produces garbage that looks plausible, the GPU version of the uninitialized-read bug.

- **synccheck:** detects invalid use of synchronization primitives, such as a `__syncthreads` that not all threads in a block reach, which is undefined behavior that can hang or corrupt.

```
# check a CUDA program for memory errors (default tool)
compute-sanitizer --tool memcheck ./matmul_cuda

# check the tiled kernel for shared-memory races
compute-sanitizer --tool racecheck ./matmul_cuda

# check for reads of uninitialized device memory
compute-sanitizer --tool initcheck ./matmul_cuda
```

bash

Run on our `matmul_cuda` benchmark, `compute-sanitizer` should report clean, because the kernels are correct: the bounds checks guard against out-of-range threads, the `__syncthreads` calls properly order the shared-memory accesses, and the inputs are fully initialized. That clean report is the GPU equivalent of the zero-errors Memcheck output from the core book, and it is the reassurance you want before trusting any GPU computation. For Dorado, you would run `compute-sanitizer` if you ever modified or wrote a custom kernel, for instance implementing the Signal-to-Base cache as a custom CUDA kernel; before trusting that the optimized kernel produces the same basecalls as the original, `racecheck` and `memcheck` would verify it has no memory or synchronization bugs. Correctness first, on the GPU exactly as on the CPU.

23.4 Practical GPU Monitoring on the Cluster

The deep profilers are for analysis. For day-to-day monitoring on the LUNA cluster, especially watching whether Dorado is actually using the GPU well during a long run, there is a tier of lightweight tools that the core book did not mention but that you will use constantly. Their unique selling point is immediacy: a single command shows you the live GPU state with no setup and no overhead.

- **nvidia-smi:** the baseline, always present with the driver. It shows each GPU's utilization, memory usage, temperature, power, and the processes using it. The `nvidia-smi dmon` mode prints a live per-second stream of utilization and memory throughput, the quickest way to see if Dorado is keeping the GPU busy or leaving it idle.
- **nvitop and nvtop:** interactive TUI monitors, like a GPU-aware top. They show utilization, memory, and per-process GPU usage in a live updating display, far more readable than raw `nvidia-smi` for watching a run unfold. `nvitop` in particular gives a clear at-a-glance view of whether the GPU is saturated.
- **gpustat:** a minimal one-line-per-GPU summary, useful for quickly checking the state of all GPUs on a multi-GPU node before launching work.
- **NVIDIA DCGM (Data Center GPU Manager):** the cluster-grade tool, designed for monitoring GPUs across a data center. Its `dcgmi` command and its monitoring daemon collect detailed health and utilization metrics over time, which is the right tool for understanding GPU behavior across a multi-node LUNA job rather than a single device.

The practical workflow on LUNA is to launch Dorado and watch `nvitop` or `nvidia-smi dmon` in another terminal. If the GPU utilization sits well below a hundred percent, the GPU is being starved, and the cause is upstream: data loading, host-device transfer, or insufficient batching, exactly the host-side bottlenecks the core book's streams chapter addressed. If utilization is pegged near a hundred percent, the GPU is well fed and the optimization target is inside the kernels, which sends you to Nsight Compute. This live utilization check is the cheapest possible triage for a GPU workload, the GPU analogue of glancing at the CPU percentage in `/usr/bin/time`, and it tells you in seconds whether Dorado's problem is on the GPU or upstream of it.

How This Connects to Your Project

This chapter closes the most important Dorado gap. The core book's Nsight analysis identified the hot kernels and classified them on the roofline, but it could not name the model operators behind them. The framework profilers do: via the no-code-change path of capturing LibTorch's NVTX operator ranges in Nsight Systems, or via the PyTorch profiler and Holistic Trace Analysis where a compatible trace can be produced. The result is a report that says not just "a GEMM kernel is memory-bound" but "the attention projection in the encoder is memory-bound," which is the difference between an abstract finding and an actionable one.

`compute-sanitizer` becomes essential the moment you implement the Signal-to-Base cache as a custom kernel: it verifies the optimized kernel is free of memory and race bugs before you trust that it produces identical basecalls, the GPU correctness-before-performance discipline. And the monitoring tools are your daily LUNA triage: `nvitop` tells you in seconds whether a Dorado run is GPU-saturated or starved, routing you either into Nsight Compute for kernel work or into the host-side data-loading optimizations. The caveat throughout: the exact integration of the PyTorch profiler and HTA with an unmodified Dorado binary depends on Dorado's build and profiling support, so verify the NVTX and Kineto paths against your version; the NVTX-in-Nsight-Systems route is the most reliable for an unmodified binary.

Common Pitfall

Stopping at kernel-level analysis for a framework model. Nsight names the kernel, not the operator. For Dorado you need operator-level attribution to know which model layer to optimize, which requires the framework profilers or NVTX operator ranges.

Assuming `torch.profiler` drives a C++ LibTorch binary like Dorado directly. It is a Python interface; for Dorado the reliable path is capturing LibTorch's NVTX ranges in Nsight Systems, with Kineto-from-C++ as a possibility to verify against the build.

Trusting a modified GPU kernel without `compute-sanitizer`. GPU memory errors and races often corrupt silently rather than crashing. Run `memcheck` and `racecheck` before trusting any custom or modified kernel, exactly as `Memcheck` on the CPU side.

Profiling deeply before checking GPU utilization. A thirty-second glance at `nvitop` tells you whether the GPU is even busy. If it is starved, the problem is host-side and no kernel analysis will help; check utilization first.

What We Just Learned

Nsight sees kernels; deep-learning profilers see operators. For a framework model like Dorado, you must map hot kernels back to the model operators that launched them, which is the operator-level attribution Nsight alone does not provide.

The PyTorch profiler (backed by Kineto, using CUPTI) correlates framework operator events with GPU kernel activity. For C++ LibTorch like Dorado, the reliable no-code-change path is capturing LibTorch's NVTX operator ranges in Nsight Systems; the Kineto-from-C++ path is worth verifying against Dorado's build.

Holistic Trace Analysis consumes profiler traces and computes higher-level diagnostics: the temporal breakdown (compute versus idle, quantifying the timeline gaps) and the kernel-by-operator ranking (the actionable attribution).

compute-sanitizer is GPU correctness, the analogue of Memcheck and Helgrind: memcheck for invalid accesses, racecheck for shared-memory races, initcheck for uninitialized reads, synccheck for synchronization misuse. Essential before trusting any custom or modified kernel.

Lightweight GPU monitoring (nvidia-smi dmon, nvidia-smi top, nvidia-smi top, gpustat, and cluster-grade DCGM) gives immediate live utilization. A glance tells you whether the GPU is saturated (optimize the kernel) or starved (optimize host-side data loading), the cheapest GPU triage.

For the project, these tools name the model operators behind Dorado's hot kernels, verify the correctness of any custom Signal-to-Base cache kernel, and provide daily LUNA triage. Verify the framework-profiler integration against Dorado's specific build; the NVTX path is most reliable for an unmodified binary.

Before You Move On

You can now connect Dorado's hot kernels to its model operators, verify GPU correctness, and triage GPU utilization on the cluster, closing the largest gap in the core book's GPU analysis.

The next chapter turns back to the CPU and the Kraken-2 Xeon, covering the automated and commercial tools beyond VTune: Intel Advisor, which automates the roofline you built by hand and advises on vectorization, and Intel PCM, which reads the uncore memory-bandwidth counters perf cannot easily reach. It then tours the HPC cluster profilers you would need if you ever scale the pipeline across multiple LUNA nodes, the world of MPI-aware, at-scale, low-overhead profiling the single-machine core book did not enter.

CHAPTER 24

Intel and HPC Power Tools

The core book gave you VTune, Intel's flagship CPU profiler, with its Top-Down Methodology. VTune is not the only Intel performance tool, and Intel is not the only vendor, and a single machine is not the only place code runs. This chapter covers three things the core book did not: the other Intel power tools that complement VTune, chiefly Intel Advisor for automated roofline and vectorization analysis and Intel PCM for the uncore counters perf cannot reach; the equivalent tools for other vendors; and the HPC cluster profiling suite you would need if you ever scaled the Nanopore pipeline beyond a single LUNA node. The Intel tools apply directly to Kraken-2 on the lab's Xeon. The HPC suite is forward-looking, relevant when the work outgrows one machine.

24.1 Intel Advisor: Automating the Roofline

The roofline model was a centerpiece of the core book. You computed arithmetic intensity by hand, measured peak bandwidth with LIKWID, and placed your kernel on the roof manually. Intel Advisor automates this entire process. Its unique selling point is that it generates the roofline for you, automatically, by instrumenting your program to measure both its floating-point throughput and its memory traffic, then plotting every loop and function as a dot on the roofline against the machine's measured ceilings. What took a section of arithmetic and a separate LIKWID run, Advisor produces as a single annotated chart with every hotspot placed.

Beyond the roofline, Advisor's second purpose is vectorization analysis, and this is genuinely valuable for numerical CPU code like Kraken-2's hash computations or any tight loop. It examines each loop and reports whether the compiler vectorized it, and if not, why not: a data dependency, an unaligned access, a function call that blocked vectorization, a pattern the compiler could not prove safe. Vectorization, using the SIMD units to process multiple elements per instruction, is one of the largest single-core speedups available, and a loop that fails to vectorize is often leaving a factor of four or eight of throughput on the table. Advisor tells you exactly which loops failed and why, and often suggests the fix, a pragma, an alignment, a restructuring. This is a question neither perf nor VTune answers directly, and it is the kind of finding that turns a mediocre loop into a fast one.

```
source /opt/intel/oneapi/advisor/latest/env/vars.sh

# survey: find the hotspots and their vectorization status
advisor --collect=survey --project-dir=adv ./matmul_naive

# roofline: measure FLOPs and traffic, generate the roofline chart
advisor --collect=roofline --project-dir=adv ./matmul_naive

# view the results (GUI) or generate a command-line report
advisor --report=roofline --project-dir=adv
```

bash

Run on our matmul benchmarks, Advisor would place the naive variant low under the memory-bandwidth ceiling, confirming memory-boundness automatically, and place the ikj and tiled variants higher and rightward as their arithmetic intensity improves, drawing the same roofline migration the core book described but without the manual arithmetic. For Kraken-2 on the Xeon, Advisor's roofline places the hash-lookup loop on the roof automatically, and its vectorization report tells you whether the lookup's inner computations are vectorized or whether they are blocked by the unpredictable data-dependent access pattern, which for a hash probe they very likely are. That vectorization verdict is information the core book's tools did not surface, and it directly informs whether there is any SIMD headroom in the lookup or whether the random-access latency dominates so completely that vectorization is irrelevant.

Key Insight

Advisor automates two things the core book did by hand or not at all: the roofline (drawn automatically with every loop placed, no manual arithmetic or separate LIKWID run) and vectorization analysis (which loops use the SIMD units, and why the others do not). The roofline automation is a convenience; the vectorization analysis is a genuinely new capability, because a loop that fails to vectorize silently wastes a large factor of single-core throughput, and neither perf nor VTune tells you why a loop did not vectorize. On the Intel Xeon of the lab machine, Advisor is the natural complement to VTune for any tight numerical loop.

24.2 Intel PCM: The Uncore Counters

There is a category of hardware counter that perf struggles to read: the uncore counters. The CPU core has counters for instructions, cycles, and cache events, which perf reads well. But the memory controllers, the inter-socket links, the shared L3 slices, these live in the uncore, the part of the chip outside the cores, and their counters are harder to access and less well exposed through perf's interface. Intel PCM, the Performance Counter Monitor, is an open-source tool built specifically to read the uncore. Its unique selling point is visibility into the memory subsystem at the socket and channel level: the actual bytes flowing through each memory channel, the L3 occupancy, the inter-socket traffic, none of which the core-level counters show.

The most useful PCM tool for our purposes is pcm-memory, which reports the live memory bandwidth per channel and per socket. Where LIKWID measured the peak bandwidth with a benchmark and perf gave you cache-miss counts, pcm-memory tells you the actual bandwidth your program is consuming, in real time, broken down by the physical memory channels. For Kraken-2 this directly measures how much of the memory subsystem the hash lookups are saturating, and the per-channel breakdown reveals whether the traffic is balanced across channels or concentrated, which affects whether the program is truly bandwidth-bound or could do better with more parallelism across channels.

```
# clone and build Intel PCM (github.com/intel/pcm), then:

# live per-channel and per-socket memory bandwidth
sudo pcm-memory 1          # update every 1 second

# overall uncore + core summary
sudo pcm 1                 # cycles, IPC, L3 hits, memory bandwidth
```

bash

Running `pcm-memory` alongside Kraken-2 gives you the achieved memory bandwidth directly, which combined with LIKWID's measured peak gives the fraction-of-peak the report needs, now from a real-time uncore measurement rather than an inference from cache-miss counts. If `pcm-memory` shows Kraken-2 saturating the channels, it is genuinely bandwidth-bound and the fix is less traffic. If it shows the channels far from saturation despite a high miss rate, the program is latency-bound, waiting on scattered individual accesses rather than saturating the bus, which is the random-probe pattern the Hot-K-mer cache targets, and the same bandwidth-versus-latency distinction VTune's Top-Down made and that the core book's LIKWID chapter described. PCM confirms it with a direct uncore measurement. A note on access: PCM, like `perf`, needs permission to read the counters and may need the `msr` kernel module loaded; it works on Intel hardware like the lab Xeon and is less applicable elsewhere.

24.3 Other Vendors and a Modern `gprof`

Briefly, the equivalents outside the Intel-and-`perf` world, so you recognize them when the hardware changes.

- **AMD uProf:** AMD's answer to VTune, a CPU profiler tuned for AMD processors with its own Top-Down-style analysis, power profiling, and the ability to read AMD's performance counters that Intel-specific tools cannot interpret. If you ever profile on AMD hardware, uProf is the VTune-equivalent to reach for, since VTune's Intel-specific TMA does not fully apply there.
- **Intel Inspector:** a correctness tool, the commercial analogue of Valgrind's Memcheck and Helgrind, detecting memory errors and threading errors with lower overhead than Valgrind. Where the core book used Memcheck, Inspector is the Intel-ecosystem alternative, and Chapter 25's sanitizers are the faster open-source alternative.
- **gprofng:** Oracle's modern replacement for `gprof`, now part of GNU binutils. Its unique selling point is being everything `gprof` should have been: a sampling profiler that handles multithreaded programs correctly, sees inlined functions, needs no `-pg` recompilation, and reads hardware counters, fixing every one of the four `gprof` limitations the core book catalogued. If you want a `gprof`-style workflow without `gprof`'s flaws, `gprofng` is the modern choice.

24.4 The HPC Cluster Profiling Suite

The entire core book was single-machine. The moment you scale the Nanopore pipeline across multiple LUNA nodes, communicating over a network with MPI, a new class of profiling questions

appears that none of the single-machine tools answer: how much time is spent in communication versus computation, is the work balanced across nodes, where are the synchronization bottlenecks across the whole job. The HPC community built a suite of tools for exactly this, and while they are out of scope for single-machine profiling, they are the right tools if the project ever runs at scale. I will tour them with their unique selling points, since going deep on all of them would be a book in itself, and they are forward-looking for this project rather than immediately needed.

- **HPCToolkit:** a sampling-based profiler from Rice University, designed for large parallel programs, with GPU support. Its unique selling point is low-overhead call-path profiling that scales to thousands of cores and attributes time across the full calling context, with a viewer (hpcviewer) that navigates the call tree at scale. It is the closest thing to a perf-style sampling profiler built for HPC scale, and notably it also profiles GPU activity, making it relevant for a GPU pipeline run across nodes.
- **TAU (Tuning and Analysis Utilities):** a comprehensive, long-established HPC profiling and tracing framework. Its unique selling point is breadth: it instruments and profiles across languages and parallel models (MPI, OpenMP, CUDA) with a single framework, producing both profiles and traces. It is the Swiss-army knife of HPC profiling.
- **Score-P with Scalasca and Vampir:** an instrumentation and measurement framework (Score-P) feeding two analysis tools, Scalasca, which automatically detects communication and synchronization bottlenecks across an MPI job, and Vampir, which visualizes the full parallel timeline. Their combined unique selling point is automatic detection of the classic MPI pathologies (late senders, imbalanced collectives, wait states) that are invisible without a cross-node view.
- **Caliper:** a lightweight instrumentation library from Lawrence Livermore that you embed in your application to mark regions and collect performance data with low overhead, designed to be always available in the application rather than bolted on. Its unique selling point is in-application, production-grade instrumentation under the developer's control.
- **Arm Forge / MAP:** a commercial HPC profiler and debugger (formerly Allinea), whose unique selling point is an exceptionally usable interface for profiling MPI programs at scale, with a low-overhead sampling profiler (MAP) and an integrated parallel debugger (DDT). Where the open-source HPC tools can be cumbersome, Arm's offering is polished.

The common thread is that these tools answer the cross-node, communication-aware questions that single-machine profiling cannot, and they are built to scale to job sizes where attaching a heavyweight tool to every process would be infeasible, hence the emphasis on low-overhead sampling and on automatic bottleneck detection. For the Nanopore pipeline today, running on a single LUNA node or the sauron machine, they are unnecessary; the single-machine toolkit suffices. But if Dorado basecalling or Kraken-2 classification ever scales across nodes, HPCToolkit (for its GPU support and sampling scale) or the Score-P family (for MPI bottleneck detection) would be the entry points, and recognizing which tool answers which scaling question is what this tour is for.

How This Connects to Your Project

For Kraken-2 on the lab's Intel Xeon, Intel Advisor and PCM are immediately useful additions to the core book's CPU toolkit. Advisor automates the roofline you built by hand and, more valuably, tells you whether the hash-lookup loop vectorizes or why it does not, a question the core book's tools left open. PCM's pcm-memory directly measures the achieved memory bandwidth per channel, confirming with a real uncore measurement whether Kraken-2 is bandwidth-bound (channels saturated) or latency-bound (channels idle despite high miss rate), the distinction that determines whether the Hot-K-mer cache helps by reducing traffic or by improving locality.

The HPC suite is forward-looking. As long as the pipeline runs on one node it is not needed. But the LUNA cluster is multi-node, and if you ever distribute Dorado basecalling or Kraken-2 classification across nodes, the single-machine tools go blind to the cross-node picture, and HPCToolkit or Score-P with Scalasca would become the right tools for diagnosing communication and load-balance bottlenecks at scale. Knowing they exist means that when the work outgrows one machine, you will know the profiling world has tools built exactly for that transition, rather than trying to stretch single-machine tools past their design. The caveat: Advisor and PCM are Intel-focused and need counter access; verify they run on the specific lab Xeon with the necessary permissions before relying on them for the report.

Common Pitfall

Computing the roofline entirely by hand when Advisor would draw it automatically with every loop placed. The hand computation builds understanding (which is why the core book did it), but for routine work Advisor is faster and less error-prone.

Overlooking vectorization. A loop that fails to vectorize silently wastes a large factor of single-core throughput, and only a vectorization analyzer like Advisor tells you which loops failed and why. perf and VTune do not surface this directly.

Inferring achieved memory bandwidth from cache-miss counts when PCM measures it directly from the uncore. The direct measurement settles the bandwidth-versus-latency question that an inference leaves ambiguous.

Reaching for HPC cluster tools on a single-machine workload, or conversely trying to stretch single-machine tools across a multi-node job. Match the tool's scope to the job's scope: single-machine tools for one node, the HPC suite for many.

What We Just Learned

Intel Advisor automates the roofline (drawing it with every loop placed, no manual arithmetic) and analyzes vectorization (which loops use the SIMD units and why the others fail), the latter a genuinely new capability the core book's tools did not provide. On the lab Xeon it complements VTune for any tight numerical loop.

Intel PCM reads the uncore counters perf struggles with, and pcm-memory reports live per-channel memory bandwidth, directly measuring whether a workload saturates the memory subsystem (bandwidth-bound) or leaves it idle despite high miss rates (latency-bound).

Other-vendor and modern equivalents: AMD uProf is VTune for AMD, Intel Inspector is a commercial Valgrind, and gprofng is a modern gprof that fixes all four of gprof's limitations (threads, inlining, recompilation, hardware counters).

The HPC suite answers cross-node, communication-aware questions single-machine tools cannot: HPCToolkit (low-overhead call-path sampling at scale, with GPU support), TAU (broad multi-model framework), Score-P with Scalasca and Vampir (automatic MPI bottleneck detection and timeline visualization), Caliper (in-application instrumentation), and Arm MAP (polished commercial HPC profiling).

For the project, Advisor and PCM are immediately useful for Kraken-2 on the Xeon, confirming the bandwidth-versus-latency diagnosis and the vectorization headroom. The HPC suite is forward-looking, needed only if the pipeline scales across multiple LUNA nodes. Verify Intel-tool counter access on the lab machine first.

Before You Move On

You now have the automated roofline and vectorization analysis for Kraken-2's Xeon, the uncore bandwidth measurement that settles the bandwidth-versus-latency question, and a map of the HPC suite for if the work ever scales across nodes.

The next chapter addresses correctness and static analysis: the compiler sanitizers that catch the same bugs as Valgrind's Memcheck at a fraction of the overhead, making them practical to run on full-scale inputs where Valgrind is too slow; the fast heap profiler heaptrack; and the static instruction-throughput analyzers llvm-mca and uiCA that predict a loop's performance without running it at all. These sharpen both the correctness checking and the prediction side of the toolkit.

CHAPTER 25

Faster Correctness, Static Analysis

The core book gave you Valgrind's Memcheck and Helgrind for correctness, with one large caveat repeated throughout: Valgrind is ten to fifty times slower than native, too slow to run on a full-scale workload. There is a faster way to catch most of the same bugs, and the core book did not cover it: the compiler sanitizers, which instrument your code at compile time rather than rewriting it at run time, catching memory and threading errors at a fraction of Valgrind's overhead. This chapter covers them, plus the fast heap profiler heaptrack, plus a genuinely different idea, static analysis that predicts a loop's performance without running it at all, using tools like llvm-mca. The theme is doing more with less overhead: faster correctness checking and prediction that needs no execution.

25.1 The Compiler Sanitizers

A sanitizer is correctness instrumentation that the compiler bakes into your binary when you pass a flag like `-fsanitize=address`. Because the compiler inserts the checks at compile time, with knowledge of the program's structure, the instrumentation is far more efficient than Valgrind's run-time rewriting of an already-compiled binary. The trade-off is that you must recompile with the sanitizer flag, where Valgrind runs any binary unchanged. But the recompilation buys you a typical slowdown of around two times rather than Valgrind's ten to fifty, which is the difference between a check you can run on a full-scale input and one you can only run on a toy input. For Kraken-2, where Valgrind's Memcheck was impractical on the real database, a sanitized build is practical, and that practicality is the whole point.

There are several sanitizers, each catching a class of bug, and they map closely onto the Valgrind tools the core book covered, so you already understand what they find.

- **AddressSanitizer (ASan), `-fsanitize=address`:** catches out-of-bounds accesses (heap, stack, and global), use-after-free, use-after-return, and double-free, the same addressability bugs as Valgrind's Memcheck, at roughly 2x slowdown instead of Memcheck's much higher cost. This is the workhorse and the one you reach for first.
- **UndefinedBehaviorSanitizer (UBSan), `-fsanitize=undefined`:** catches undefined behavior the C standard does not define: signed integer overflow, invalid shifts, null-pointer dereferences, misaligned pointers, type confusion. This is a class of bug Valgrind largely does not catch, so UBSan is genuinely additive, not just faster.
- **ThreadSanitizer (TSan), `-fsanitize=thread`:** catches data races, the same unsynchronized-concurrent-access bugs as Valgrind's Helgrind, and generally with fewer false positives and clearer reports. For threaded code like the OpenMP matmul or multi-threaded Kraken-2, TSan is the modern race detector of choice.
- **MemorySanitizer (MSan), `-fsanitize=memory`:** catches reads of uninitialized memory, the definedness bugs Memcheck's `--track-origins` targets. MSan is powerful but stricter to use (it

needs the whole program, including libraries, instrumented to avoid false positives), so it is the most specialized of the four.

```
# AddressSanitizer: the first-reach correctness check, ~2x slowdown
gcc -O1 -g -fsanitize=address -o matmul_asan matmul_naive.c
./matmul_asan                # reports errors live, with stacks

# UndefinedBehaviorSanitizer: catches what Valgrind misses
gcc -O1 -g -fsanitize=undefined -o matmul_ubsan matmul_naive.c
./matmul_ubsan

# ThreadSanitizer: data races in the OpenMP version
gcc -O1 -g -fsanitize=thread -fopenmp -o matmul_tsan matmul_omp.c
OMP_NUM_THREADS=8 ./matmul_tsan
```

bash

A few practical notes. You typically build sanitized at `-O1` with `-g`: enough optimization to be representative, enough debug info for clear reports, and not so much optimization that the sanitizer's checks get tangled. You cannot combine ASan and TSan in one build (they conflict), so you run separate sanitized builds for memory and for threading. And the sanitizers report errors live as the program runs, printing the offending stack immediately, rather than at exit, which makes the first error easy to find and fix. Run on our clean benchmarks they report nothing, the same all-clear as Valgrind, but at a speed that makes running them routine rather than a special occasion.

Key Insight

Sanitizers and Valgrind catch overlapping bugs by opposite mechanisms. Valgrind rewrites any binary at run time, no recompilation, but 10 to 50x slowdown. Sanitizers require a recompile but run at roughly 2x, fast enough for full-scale inputs. The practical rule: use sanitizers as your routine, fast, full-scale correctness check during development, since you control the source and can recompile; reserve Valgrind for the cases where you cannot recompile (a third-party binary) or need its specific capabilities (Cachegrind's cache simulation, which sanitizers do not do). For Kraken-2, where Memcheck was too slow for the real database, a sanitized build makes correctness checking at full scale finally practical, and UBSan adds a bug class Valgrind misses entirely.

25.2 heaptrack: Fast Heap Profiling

The core book used Massif for heap profiling, and Massif, being a Valgrind tool, carries Valgrind's overhead. heaptrack, from KDAB, is a faster heap profiler with the same purpose: tracking where heap memory is allocated, how it grows over time, and which call sites are responsible. Its unique selling point is low overhead combined with a polished analysis GUI, making it practical to profile heap behavior on realistic workloads where Massif would be too slow, and pleasant to explore the results.

heaptrack records every allocation with its stack and timestamp, then its GUI shows the heap growth over time, the peak consumers ranked by call site, the allocation count (to find code that allocates too frequently, a different problem from allocating too much), and the temporary

allocations that are freed almost immediately (a common, wasteful pattern). For a program with significant heap activity, heaptrack gives the same insights as Massif but fast enough to run routinely and with a far better interface for finding the answer.

```
# record heap activity (low overhead)
heaptrack ./prog

# analyze (GUI) or print a summary
heaptrack_print heaptrack.prog.PID.gz | head -40
# or open heaptrack.prog.PID.gz in the heaptrack GUI
```

bash

The honest caveat for Kraken-2 is the same one the core book raised about Massif: Kraken-2's 180 GB database is memory-mapped, not malloc-allocated, so a heap profiler that tracks malloc-family allocations sees the database only partially or not at all. heaptrack, like Massif, primarily tracks heap allocations, so for the database itself the page-level view (Massif's `--pages-as-heap`, or the eBPF page-fault tracing from Chapter 22) is the right tool. Where heaptrack shines for Kraken-2 is the non-mmap heap: the per-read working buffers, the result structures, any allocation the classification loop does per read. If that per-read heap activity is significant, heaptrack finds it fast, and the Hot-K-mer cache itself, being a malloc-allocated structure, would show up clearly in heaptrack, letting you confirm its memory footprint precisely.

25.3 Static Analysis: Predicting Performance Without Running

Everything in both books so far has been dynamic: you run the program and measure. There is a genuinely different approach, static analysis, that predicts a loop's performance by analyzing its machine code without executing it. The tools model the CPU's execution units, latencies, and throughputs, and compute how many cycles a loop iteration should take in the ideal case where everything is in cache. Their unique selling point is prediction without execution: they tell you the best-case throughput of a hot loop, which, compared against the measured performance, reveals how much of your slowness is the loop's instruction-level structure versus memory stalls.

The main tool is **llvm-mca**, the LLVM Machine Code Analyzer. You give it a snippet of assembly, the hot loop, and it simulates that loop running on a model of a chosen CPU, reporting the predicted cycles per iteration, the throughput bottleneck (which execution port is the limit), and the instruction-level parallelism achieved. A related tool, **uiCA**, the uops.info Code Analyzer, does similar throughput prediction with a very accurate model of Intel front-end behavior. Both answer the question: ignoring memory, how fast could this loop go, and what limits it.

```
# extract the hot loop's assembly and analyze its ideal throughput
gcc -O2 -S matmul_ikj.c -o matmul_ikj.s
# (isolate the inner-loop assembly into loop.s, then:)
llvm-mca -mcpu=skylake loop.s

# llvm-mca reports, e.g.:
#   Iterations:      100
#   Total Cycles:    ...
#   Block RThroughput: 2.0    <- cycles per iteration, ideal case
#   bottleneck: the FP add port
```

bash

Here is why this is useful and how it connects to the roofline thinking. `llvm-mca` tells you the ideal cycles per iteration assuming perfect cache behavior, the loop's intrinsic compute cost. `perf` tells you the actual cycles per iteration including all the memory stalls. The gap between them is the memory penalty. For our naive `matmul`, `llvm-mca` would predict a small ideal cost per iteration (a couple of cycles for the FMA), while `perf` measured a large actual cost (dozens of cycles per iteration from the cache misses), and that enormous gap is the precise quantification of how much the memory behavior, not the arithmetic, dominates. For the `ikj matmul`, the `llvm-mca` ideal and the `perf` actual would be close, confirming the loop is near its compute-bound ceiling. Static analysis gives you the compute-bound denominator, the best the loop could do, against which the measured performance reveals the memory cost, a different route to the same compute-versus-memory split the roofline draws.

Two adjacent tools worth naming. The compiler's own vectorization reports, enabled with `-fopt-info-vec` on `gcc` or `-Rpass=loop-vectorize` on `clang`, tell you statically which loops the compiler vectorized and which it did not, the same vectorization question Intel Advisor answers dynamically, available for free from the compiler with no separate tool. And Compiler Explorer, the web tool at godbolt.org, lets you see the exact assembly the compiler generates for a snippet across compilers and flags, invaluable for understanding what your source actually became and whether an optimization you expected (vectorization, inlining, a fused multiply-add) actually happened. Reading the generated assembly is the ground truth of what the CPU will execute, and Compiler Explorer makes it effortless.

How This Connects to Your Project

For Kraken-2's C code, the sanitizers are the immediately practical addition. Where the core book noted Memcheck was too slow for the real database, an AddressSanitizer build runs at roughly 2x and can validate the code on full-scale input, and a ThreadSanitizer build checks the multi-threaded classification for races at a usable speed. When you implement the Hot-K-mer LRU cache, the sanitizers are how you verify the new code is memory-safe and race-free on realistic input before trusting its results, the faster complement to the Valgrind checks the core book prescribed, and UBSan adds an integer-overflow and undefined-behavior check Valgrind does not provide. heaptrack confirms the cache's heap footprint precisely.

Static analysis sharpens the bottleneck argument. Running llvm-mca on the hash-lookup loop's hot assembly gives the ideal, memory-free cycles per iteration; comparing that to perf's measured cycles per iteration quantifies exactly how much of the lookup's cost is the random-access memory latency versus the intrinsic computation. If the gap is enormous, as it will be for a random-probe lookup, that is a precise, static-versus-dynamic demonstration that the bottleneck is memory, not arithmetic, strengthening the case for a locality optimization. And reading the generated assembly in Compiler Explorer confirms whether the compiler vectorized or fused anything in the lookup, the same headroom question Advisor answers, available with no separate tool. The caveat: llvm-mca models the ideal case and a specific CPU model; its prediction is the best-case ceiling, not a measurement, and the comparison to real perf numbers is what makes it informative.

Common Pitfall

Using Valgrind's Memcheck on a full-scale input it is too slow for, when a sanitized build runs at 2x and makes full-scale correctness checking practical. Use sanitizers as the routine check; reserve Valgrind for unrecompilable binaries or its unique capabilities.

Combining AddressSanitizer and ThreadSanitizer in one build. They conflict. Build and run them separately, one for memory bugs, one for races.

Treating a heap profiler as the tool for Kraken-2's mmap'd database. heaptrack and Massif track heap allocations; the memory-mapped database needs page-level tracking (Massif --pages-as-heap or eBPF). Use heaptrack for the non-mmap per-read heap activity.

Reading llvm-mca's prediction as a measurement. It is the ideal best-case throughput for a chosen CPU model, assuming perfect cache behavior. Its value is the comparison against real perf numbers, which reveals the memory penalty as the gap between ideal and actual.

What We Just Learned

Compiler sanitizers instrument at compile time, catching overlapping bugs with Valgrind at roughly 2x slowdown instead of 10 to 50x, fast enough for full-scale inputs. The trade-off is needing to recompile.

The four sanitizers map onto the Valgrind tools: AddressSanitizer for out-of-bounds and use-after-free (the workhorse), UndefinedBehaviorSanitizer for undefined behavior Valgrind misses (genuinely additive), ThreadSanitizer for data races (the modern Helgrind), MemorySanitizer for uninitialized reads (the strictest). ASan and TSan cannot share a build.

heaptrack is a fast, GUI-equipped heap profiler, the low-overhead alternative to Massif, but like Massif it tracks heap allocations not mmap, so for Kraken-2 it suits the per-read heap activity and the cache's footprint, not the mapped database.

Static analysis predicts a loop's performance without running it. llvm-mca and uiCA model the CPU and report the ideal cycles per iteration and the throughput bottleneck, the compute-bound ceiling assuming perfect cache.

The gap between llvm-mca's ideal cycles and perf's measured cycles is the precise memory penalty, a static-versus-dynamic route to the compute-versus-memory split. Compiler vectorization reports and Compiler Explorer show statically what the compiler did to your loop.

For the project, sanitizers make full-scale correctness checking of Kraken-2 and the Hot-K-mer cache practical where Memcheck was too slow, and llvm-mca quantifies the lookup's memory penalty as the gap between its ideal and measured throughput. Treat static predictions as ceilings, not measurements.

Before You Move On

You now have fast, full-scale correctness checking via the sanitizers, fast heap profiling via heaptrack, and the static-analysis route to the compute-bound ceiling via llvm-mca, all lower-overhead or zero-overhead complements to the dynamic tools of the core book.

The final chapter of this companion volume sweeps up the remaining categories: I/O profiling for the disk behavior of reading pod5 signal and the Kraken database, Python profilers for any orchestration glue, always-on continuous profilers for production-style monitoring, and the genuinely novel causal profiler Coz that tells you which optimization would actually speed up the whole pipeline. It closes with the extended decision map that folds every tool from both volumes into one framework and a recommended extended toolkit for the Nanopore project.

CHAPTER 26

I/O, Python, Continuous, the Map

This final chapter sweeps up the remaining categories the core book did not cover, then folds everything from both volumes into one extended decision map and a recommended toolkit for the Nanopore project. The categories here are I/O profiling, which matters because both halves of the pipeline read large files from disk; Python profiling, which matters if any of the orchestration glue is Python; continuous profiling, the always-on production style; and one genuinely novel idea, causal profiling, which answers a question no other tool does: not where time goes, but which optimization would actually speed up the whole pipeline. Most of these are a tour with the unique selling point named; a few are project-relevant enough to dwell on. Then the map, which is the real payoff of the companion volume: a single framework that places every tool from both books.

26.1 I/O Profiling

Every tool in the core book looked at CPU and memory. None looked carefully at disk I/O, yet both halves of the pipeline are I/O-touching: Kraken-2 reads its database and input reads from disk, and Dorado reads pod5 signal files, often large, and the disk feeding that signal can starve the GPU, the exact host-side bottleneck the Nsight Systems timeline gaps hinted at. I/O profiling answers how much time goes to disk, how fast the disk delivers, and whether I/O is the hidden bottleneck.

- **iostat and iotop:** the quick views, building on the core book's system-tools chapter. `iostat -x` shows per-device utilization and the average wait time per I/O request; `iotop` shows which processes are doing the I/O right now, the disk analogue of `top`. These are the first-reach disk triage.
- **blktrace and blkparse:** the detailed block-layer tracer, recording every I/O request to a device with timing, for understanding exactly how the disk requests are issued and completed. Its unique selling point is request-level block-I/O detail, the ground truth of what the storage layer did.
- **fio (Flexible I/O tester):** not a profiler of your program but a benchmark of your storage, the I/O analogue of `likwid-bench`. Its unique selling point is measuring the disk's actual throughput and latency under a configurable access pattern, giving you the storage ceiling against which your program's I/O can be judged, the same fraction-of-peak discipline applied to disk.
- **Darshan:** an HPC I/O characterization tool that transparently records a program's I/O behavior, what files it accessed, how much, with what access pattern, and produces a summary report. Its unique selling point is low-overhead, transparent, after-the-fact I/O characterization at HPC scale, exactly suited to understanding how a pipeline like Dorado-plus-Kraken-2 touches storage on a cluster like LUNA.

For the project, the I/O question is concrete. If eBPF off-CPU analysis (Chapter 22) shows Kraken-2 blocking on disk reads, or if the Nsight Systems timeline shows Dorado's GPU idle while signal loads, then I/O is a real bottleneck and these tools quantify it. `fio` measures what the storage

can deliver; iostat and Darshan measure what the pipeline actually demands; the gap or the saturation tells you whether the disk is the limit. On LUNA, where storage is often a shared network filesystem rather than a local SSD, this matters more, because shared storage can be far slower and more variable than local disk, and a pipeline that assumed local-SSD speed can be badly starved on a cluster filesystem.

26.2 Python Profiling for the Pipeline Glue

Dorado is C++ and Kraken-2 is C, so the core book's CPU and GPU tools cover the heavy lifting. But real pipelines are held together with glue, and that glue is very often Python: scripts that orchestrate the stages, handle the pod5 files, parse outputs, manage the workflow. If any of that glue is slow, the C and C++ profilers cannot see it, because they profile native code, and Python's interpreted execution needs its own profilers. These are worth knowing for the parts of the pipeline that are not the compute kernels.

- **cProfile:** Python's built-in deterministic profiler, the standard-library baseline. It records every function call and its time, the Python analogue of gprof, with the same caveat of per-call overhead. Fine for a quick function-level view of a Python script.
- **py-spy:** a sampling profiler for Python whose unique selling point is that it profiles a running Python process from outside, with no code changes and no restart, even attaching to an already-running script. It produces flame graphs of Python code the same way perf does for native code, at low overhead, making it the practical choice for profiling pipeline glue in place.
- **Scalene:** a high-performance Python profiler whose unique selling point is profiling CPU, GPU, and memory together, and crucially distinguishing time spent in Python from time spent in native code called by Python. For a pipeline where Python glue calls into C extensions, Scalene tells you whether the time is in the Python or the native layer, which is exactly the question that decides whether a Python profiler or a native profiler is the right next tool.
- **memray:** a memory profiler for Python from Bloomberg, tracking Python allocations over time with a detailed GUI, the Python analogue of heaptrack. Useful if the glue's memory usage, rather than its speed, is the concern.

The honest assessment for this project is that Python profiling is a contingency, not a certainty. If the orchestration is pure shell or if the glue is trivial, you will not need these. But pod5 handling and workflow orchestration are frequently done in Python in the nanopore ecosystem, and if a profiling pass with the system tools shows significant time outside the C and C++ binaries, in a Python wrapper, then py-spy is the tool to point at it, and Scalene is the tool to determine whether that Python time is real Python work or native work the Python merely calls. Knowing these exist means you will not be stuck when the bottleneck turns out to be in the glue rather than the kernels.

26.3 Continuous and Production Profiling

Everything so far has been a profiling session: you run a tool, capture data, analyze it, stop. Continuous profiling is a different model, always on, low overhead, collecting profiles continuously from a running system so you can look back at any point and ask what was hot. Its unique selling

point is catching problems that only appear in production, under real load, at times you were not watching, which a one-off session structurally cannot. For a long-running pipeline or a service, this is increasingly the standard.

- **Pyroscope (now part of Grafana) and Parca:** open-source continuous profilers, increasingly eBPF-based, that collect CPU and memory profiles continuously across a fleet with very low overhead and store them for querying over time. Their unique selling point is always-on, fleet-wide, queryable profiling history, turning profiling from an event into a continuous data source.
- **gperftools and pprof:** Google's performance tools, a CPU profiler and the tcmalloc allocator with a built-in heap profiler, whose output is consumed by pprof, a widely used profile visualizer. The unique selling point is the pprof format and viewer, which has become a lingua franca for profiles across many languages and tools, and the lightweight always-available CPU and heap profiling that gperftools provides by linking a library.
- **Cloud and commercial continuous profilers:** Datadog, Google Cloud Profiler, and others offer continuous profiling as a managed service, the same always-on model with hosted storage and analysis. Relevant if the pipeline ever runs as a managed cloud service rather than on the lab or cluster machines.

For the Nanopore project today, running as batch jobs on the lab machine and LUNA rather than as a continuous service, continuous profiling is not on the critical path. Its relevance is forward-looking: if the pipeline ever became a standing service that processed sequencing runs continuously, a continuous profiler would catch the performance regressions and load-dependent slowdowns that a one-off profiling session during development would miss. It is the model to remember for when profiling needs to be permanent rather than occasional.

26.4 Coz: The Causal Profiler

I have saved the most conceptually interesting tool for near the end, because it answers a question every other profiler in both books cannot. Every profiler so far tells you where time is spent. None of them tells you which optimization would actually make the whole program faster, and those are different questions. A function can consume forty percent of runtime and yet optimizing it might speed the program up almost not at all, if it is off the critical path, overlapped with other work, or bounded by something else. Coz, a causal profiler from the same research lineage as Scalene, answers the optimization question directly, and its mechanism is genuinely clever.

Coz works by *virtual speedup*. To predict what would happen if you sped up a particular code region, it cannot actually speed that region up, that is the optimization you have not done yet. So instead it does the opposite in a controlled way: it slows down *everything else* by the same amount, which is mathematically equivalent to having sped up the target region, relative to the rest. By running many such experiments, virtually speeding up different regions and measuring the effect on overall throughput, Coz builds a report that says, for each region, how much faster the whole program would get if you optimized that region by a given amount. Its unique selling point is causal: it tells you the actual end-to-end payoff of optimizing each region, not just where time is spent,

which is the question you truly care about when deciding what to optimize.

For a multi-stage pipeline like Dorado-then-Kraken-2, this is exactly the right question. Suppose Dorado takes sixty percent of wall-clock time and Kraken-2 forty percent. Naively you optimize Dorado. But if the two stages overlap, or if one is bounded by I/O that optimizing the compute will not relieve, the naive choice can be wrong. Coz would tell you, by virtual-speedup experiments on each stage, how much end-to-end speedup you would actually get from optimizing each, so you spend your effort where it pays off in total throughput rather than where the time merely appears to be. This is the antidote, at the whole-pipeline level, to the radix-sort mistake from the very first chapter of the core book: it is possible to correctly identify the biggest time sink and still optimize the wrong thing, if that time sink is not on the critical path to end-to-end speedup. Coz is the only tool in either volume that measures the critical path causally. The caveat: Coz requires some integration (marking the throughput progress points it measures against) and works best on programs you can instrument lightly, so its applicability to an unmodified Dorado-Kraken-2 pipeline would need verification, but the question it answers is uniquely valuable and worth the setup where it fits.

Key Insight

Every other profiler answers where is time spent. Coz answers which optimization would speed up the whole program, by virtually speeding up each region (mechanically, by slowing everything else) and measuring the end-to-end effect. These are different questions, and the second is the one that actually governs where you should spend effort. A region can dominate runtime yet yield little end-to-end speedup if it is off the critical path or overlapped or I/O-bound. For a multi-stage pipeline, Coz is the principled way to choose which stage to optimize, the whole-system answer to the cardinal question the core book opened with.

26.5 Viewers and Frame Profilers, Briefly

A last quick tour of tools that present or capture profiles in particular ways, worth recognizing by name.

- **Hotspot (KDAB):** a GUI front end for perf, turning perf.data into interactive flame graphs and timelines, far more pleasant than the perf report TUI. The unique selling point is a good visual interface for the perf data you already collect.
- **speedscope:** a web-based flame-graph viewer that reads many profile formats, useful for sharing and exploring flame graphs in a browser without installing anything.
- **Tracy and Orbit:** frame and real-time profilers oriented toward games and latency-sensitive applications, with nanosecond-resolution instrumentation and live visualization. Their unique selling point is real-time, frame-by-frame profiling, less relevant to a batch genomics pipeline but the right tool for interactive or real-time systems.
- **Perfetto:** Google's tracing and visualization stack, used across Android and Chrome, that ingests traces from many sources (including the ftrace and the Chrome-format traces the PyTorch profiler emits) into a powerful web-based timeline viewer. Its unique selling point is a

unified, powerful trace viewer that consumes formats from across the ecosystem.

26.6 The Extended Decision Map

Now the payoff: every tool from both volumes, folded into one decision framework. The core book's decision tree answered the common questions; here is the extended version that adds the second-tier tools, organized by the question they uniquely answer.

Question	-> Tool	output
-----	-----	
(core toolkit, from the main book)		
Quick CPU/memory triage?	-> /usr/bin/time -v	
IPC and cache miss rate?	-> perf stat	
Which function is hot (on-CPU)?	-> perf record	
Exact cache counts / LLC on WSL2?	-> cachegrind	
Memory bug / heap growth / race?	-> memcheck/massif/helgrind	
GPU timeline / GPU kernel deep-dive?	-> nsight systems / compute	
Guided why-analysis / measured peak?	-> vtune / likwid	
 (this companion volume's additions)		
Where does it BLOCK (off-CPU)?	-> eBPF offcputime	
Custom trace of any kernel event?	-> bpftrace / bcc	
Which MODEL OPERATOR is the hot kernel?	-> pytorch profiler / HTA	
GPU correctness (races, bad memory)?	-> compute-sanitizer	
Live GPU utilization on the cluster?	-> nvidia-smi / DCGM	
Automated roofline + vectorization?	-> intel advisor	
Per-channel memory bandwidth (uncore)?	-> intel pcm	
Fast full-scale correctness check?	-> sanitizers (ASan/TSan)	
Ideal loop throughput without running?	-> llvm-mca / uiCA	
Disk I/O the bottleneck?	-> iostat/Darshan/fio	
Python glue slow?	-> py-spy / scalene	
Always-on production profiling?	-> pyroscope / parca	
Which optimization helps END-TO-END?	-> coz (causal)	
Profiling across multiple cluster nodes?	-> hpctoolkit / score-p	

The structure of the extended map is worth absorbing. The core tools answer the everyday questions and remain your default. The companion tools each answer a question the core tools could not: off-CPU blocked time, operator-level GPU attribution, GPU correctness, automated roofline, uncore bandwidth, fast full-scale correctness, static throughput prediction, I/O, Python, continuous, causal, and at-scale. You do not reach for the second tier routinely; you reach for it when a specific question arises that the first tier cannot answer, and the map tells you which one. Internalize the questions in the right column and the tool follows.

26.7 The Recommended Extended Toolkit for the Nanopore Project

Finally, the concrete recommendation: of everything in this companion volume, what should you actually set up and use for the Nanopore pipeline, beyond the core toolkit. This is the answer to the project-relevance question the whole volume has been building toward.

- **eBPF off-CPU analysis (Chapter 22)** for Kraken-2: to measure how much of its wall-clock time is blocked on database page faults versus computing, the blocked-time accounting the core book's on-CPU measurements could not provide, on the real database at full scale.
- **PyTorch profiler via NVTX-in-Nsight-Systems, plus compute-sanitizer (Chapter 23)** for Dorado: to attribute the hot kernels to model operators (which Transformer layer, which attention), and to verify correctness if you implement a custom Signal-to-Base cache kernel. Plus nvitop for daily LUNA utilization triage.
- **Intel Advisor and PCM (Chapter 24)** for Kraken-2 on the Xeon: Advisor for the automated roofline and the vectorization verdict on the hash loop, PCM for the per-channel bandwidth that settles bandwidth-bound versus latency-bound directly.
- **Compiler sanitizers and llvm-mca (Chapter 25)** for the C code: ASan and TSan for fast full-scale correctness of Kraken-2 and the Hot-K-mer cache, llvm-mca to quantify the lookup's memory penalty as the gap between ideal and measured throughput.
- **I/O profiling and Coz (Chapter 26)** for the pipeline: iostat or Darshan to check whether disk feeding starves either stage, and Coz to decide, causally, whether optimizing Dorado or Kraken-2 yields more end-to-end speedup.

That list, layered on top of the core book's perf, Valgrind, Nsight, VTune, and LIKWID, is a genuinely complete profiling toolkit for the pipeline, covering on-CPU and off-CPU time, kernel and operator attribution, CPU and GPU correctness, automated and manual rooflines, static and dynamic analysis, I/O, and the causal question of where optimization actually pays. Most of these you will set up once and use occasionally, when the core toolkit raises a question it cannot answer. But knowing they exist, and which question each one answers, is what separates a profiler who is stuck when the standard tools run out from one who always knows where to turn next. That, in the end, is the purpose of this companion volume: not to replace the core toolkit, but to ensure you are never out of tools when a hard question appears.

How This Connects to Your Project

This chapter completes the project's extended toolkit and answers the causal question the entire two-book effort has been circling. The core book taught you to find where time goes in Dorado and Kraken-2 and to fix the bottlenecks. This companion adds the dimensions the core book could not reach: where the program blocks rather than computes (eBPF), which model operator a hot kernel belongs to (PyTorch profiler), whether the disk starves the pipeline (I/O tools), and, with Coz, which of the two stages to optimize for the most end-to-end speedup. Set up the recommended extended toolkit, run it when the core tools raise a question they cannot answer, and you will have, for the Nanopore pipeline, a profiling practice with no blind spots: on-CPU and off-CPU, CPU and GPU, kernel and operator, correctness and performance, static and dynamic, single-machine and at-scale, and the causal critical path that tells you where the effort truly pays.

What We Just Learned

I/O profiling (iostat, blktrace, fio, Darshan) answers whether disk is the hidden bottleneck, relevant because both pipeline stages read large files and shared cluster storage can starve them. fio measures the storage ceiling; Darshan characterizes the pipeline's actual I/O.

Python profilers (cProfile, py-spy, Scalene, memray) cover the orchestration glue the native tools cannot see. py-spy attaches to a running script with no changes; Scalene distinguishes Python time from native time, deciding which profiler to use next.

Continuous profilers (Pyroscope, Parca, gperftools/pprof) collect always-on, low-overhead profiles, catching production problems a one-off session misses. Forward-looking for the project unless the pipeline becomes a standing service.

Coz is the causal profiler: by virtual speedup (slowing everything else to simulate speeding one region), it measures the end-to-end payoff of optimizing each region, answering which optimization helps overall, not just where time is spent. Uniquely valuable for choosing which pipeline stage to optimize.

Viewers and frame profilers (Hotspot, speedscope, Tracy, Orbit, Perfetto) present or capture profiles in particular ways; Perfetto notably consumes the PyTorch profiler's traces.

The extended decision map folds every tool from both volumes into one framework, each second-tier tool answering a question the core tools cannot. The recommended project toolkit adds eBPF, the PyTorch profiler and compute-sanitizer, Advisor and PCM, the sanitizers and llvm-mca, and I/O tools and Coz, on top of the core perf-Valgrind-Nsight-VTune-LIKWID foundation, for a profiling practice with no blind spots.

A Final Word

The core book gave you a complete first toolkit and the discipline to wield it: measure, do not guess; triage cheap to expensive; ground every number in a measured peak; let the data, not the intuition, name the bottleneck. This companion gave you the second tier, the tools for the questions the first toolkit cannot reach, and above all the recognition of which question each tool uniquely answers.

Together they make the Nanopore profiling effort as thorough as it can be, and they make you, for any future performance problem on any system, someone who is never out of tools and never out of the question that tool answers. The instruments were always means. The trained judgment, knowing which question to ask and which instrument answers it, is the end. You now have both tiers. Go measure, and when a question arises that the common tools cannot answer, you will know exactly where to turn.